

Week 1 - Day 3c: Functions

Welcome to today's lab! In this session, we will cover functions!

Exercise 1: Intro to Functions []

The following function computes the area of a square. Its **argument** is the `side_length` of the square, and it returns the area of the square.

```
def area_square(side_length):  
    return side_length**2
```

The cell above only **defines** the function. If you run the cell, nothing happens yet. We can **call** the function by giving it a side length as an argument, and it will **return** (give back) the area.

In the example below, we compute the area of a square of side length 3.

```
def area_square(side_length):  
    return side_length**2  
  
area_square(3)
```

You can use this function and compute the areas of squares of any side length. We use the `print` function to print the outputs.

```
def area_square(side_length):  
    return side_length**2  
  
print(area_square(3))  
print(area_square(5))  
print(area_square(10))
```

Jupyter remembers the definition of a function, so you don't need to do `def area_square(...)` each time.

```
print(area_square(2))  
print(area_square(4))  
print(area_square(12))
```

You can also assign the returned value to a variable, for example:

```
x = area_square(4)
```

What is the value of `x`?

Exercise 2: Basic Geometric Functions []

2.1 The perimeter of a square of side length a is $4a$. Fill in the code below to define a function that takes a side length as argument and returns the perimeter of a square.

```
# TODO: Your code here
def perimeter_square(side_length):
    pass
```

You can test your function by running the cell below (it checks the function output against some expected outputs).

```
assert perimeter_square(1) == 4
assert perimeter_square(2) == 8
assert perimeter_square(10) == 40

print(f"Test cases passed! []")
```

2.2 The area of an equilateral triangle of side length a is $\frac{\sqrt{3}}{4}a^2$. Write a function called `area_equilateral_triangle` that takes a side length as argument and returns the area of an equilateral triangle of that side length. Use it to compute the areas for side length 1, 2 and 10.

```
# TODO: Your code here

assert int(area_equilateral_triangle(1) * 10**5) == 43301
assert int(area_equilateral_triangle(2) * 10**5) == 173205
assert int(area_equilateral_triangle(4) * 10**5) == 692820

print(f"Test cases passed! []")
```

2.3 Write a function named `area_circle` that computes the area of a circle with a given radius. It should take a `radius` as argument and return the area.

Hint: You can approximate $\pi=3.14$

```
# TODO: Your code here

assert area_circle(1) == 3.14
assert area_circle(2) == 12.56
assert area_circle(4) == 50.24

print(f"Test cases passed! []")
```

You can also take the function output and do computations with it. For example, if you want to compute the sum of the area of two circles, you can do:

```
area_circle(2) + area_circle(3)
```

2.4 Can you use your function to compute the area of the grey shape below?

```
# TODO: Your code here
```

Exercise 3: More Geometry []

3.1 Use the functions `area_circle`, `area_equilateral_triangle` and `area_square` to compute the area of the shape below, assuming all the straight lines have length 5!

```
# TODO: Your code here
```

3.2 Write a function `perimeter_circle` that computes the **perimeter** of a circle with a given radius. It should take a `radius` as argument and return the perimeter.

Hint: Again, assume that $\pi=3.14$.

```
# TODO: Your code here
```

```
assert perimeter_circle(1) == 6.28
assert perimeter_circle(2) == 12.56
assert perimeter_circle(4) == 25.12

print(f"Test cases passed! []")
```

3.3 Functions can take multiple arguments. Let's define a function `perimeter_rectangle` that computes the perimeter of a rectangle, given its `height` and `length`.

```
# TODO: Your code here
```

```
def perimeter_rectangle(length, height):
    pass

print(perimeter_rectangle(2, 3))
print(perimeter_rectangle(3, 5))

assert perimeter_rectangle(2, 3) == 10
assert perimeter_rectangle(3, 5) == 16

print(f"Test cases passed! []")
```

Exercise 4: More Complicated Functions []

The computations inside of functions can be more complicated, and the body can consist of multiple lines. You only need **one** return statement. Consider the function below computing the area of a donut with inner radius `r_inner` and outer radius `r_outer`.

Test this function for at least 5 different donuts!

```
# TODO: Try it out for at least 5 different donuts!

def area_donut(r_inner, r_outer):
    area_outer = 3.14*r_outer**2
    area_inner = 3.14*r_inner**2
    return area_outer - area_inner
```

Exercise 5: Turning Previous Code into Functions []

5.1 Remember the code you wrote to round a number to the nearest integer:

```
n = 12.6
decimals = n - int(n)
if decimals >= 0.5:
    print(int(n)+1)
else:
    print(int(n))
```

Functions are really useful to test such code on many numbers at once!

Let's turn this into a function. Note how all of the code gets *indented* (pushed to the right), and how it is possible to have multiple `return` statements.

```
def round(n):
    # We remove the n = 12.6 here, because n is now a function
    # argument
    decimals = n - int(n)
    if decimals >= 0.5:
        return int(n)+1 # return instead of print
    else:
        return int(n)   # return instead of print

# Let's test our function!
print(round(1.2))
print(round(12.6))
print(round(0.5))
```

Test this function on at least 5 more numbers!

```
# TODO: Your code here
```

5.2 Remember the code you wrote to extract the last digit of a number:

```
n = 3027184629834
print(n % 10)
```

Turn it into a function and test it on at least 10 numbers!

```
# TODO: Your code here
def last_digit(n):
    pass
```

5.3 Remember the code from class that turns a numerical grade into a letter grade:

```
numeric_grade = 75
if numeric_grade > 90:
    letter_grade = 'A'
elif numeric_grade > 80:
    letter_grade = 'B'
elif numeric_grade > 70:
    letter_grade = 'C'
elif numeric_grade > 60:
    letter_grade = 'D'
else:
    letter_grade = 'F'
```

Write a function `get_letter_grade` that takes a numeric grade and turns it into a letter grade. Test the function for 10 different grades.

```
# TODO: Your code here
```

Exercise 6: Functions and Loop Structures []

6.1 Write a function that takes a single argument `x` and returns `True` if `x` is even.

```
# TODO: Your code here
```

6.2 Write a function that takes `x` and prints odd number between 0 and `x-1` using a `for` loop

```
# TODO: Your code here
```

6.3 Write a function that takes `x` and prints every EVEN number between 0 and `x-1` using a `while` loop

```
# TODO: Your code here
```

6.4 Using `while True:`, print the integers from 0 - 10

```
# TODO: Your code here
```

Note: If your code does not stop running and has an `[*]` on the left, it is in an infinite loop. To get out of this loop, in the top menu, select Kernel -> Interrupt, or press the black square button in the top menu bar.

6.5 Write a function that takes `x` and prints every EVEN number between 0 and `x - 1` using `while True`:

```
# TODO: Your code here
```

6.6 Write a function that takes `x` and prints every EVEN number between 0 and `x - 1` using a `for` loop, without **control flow** (i.e. no `if` statements)

```
# TODO: Your code herej
```

Exercise 7: Python Functions for DNA Sequence Analysis []

7.1 DNA has 4 bases:

1. Adenine (A)
2. Thymine (T)
3. Cytosine (C)
4. Guanine (G)

In the DNA, each base fits like a puzzle piece, with another base. So, Adenine will pair up with Thymine (A-T base pairing) and Cytosine will pair up with Guanine (C-G base pairing), and vice versa. DNA is double stranded, so if we look at one side of the strand, we can figure out the other side that matches. For example:

- `ATACGTACGATCA` # this is the strand we know
- `TATGCATGCTAGT` # this is the *complement strand*, that we can figure out.

7.1.1 Just as an exercise, fill in the complement strand to the following strands manually:

Strand: `ATCCGGGATCAA`

7.1.2 Strand: `GGCCCCATTTTC`

7.2 Given a string `x` that contains only A, C, G, and T characters, write a function called `containsA` that returns `True` if the string contains an Adenine, expressed as an `'A'`.

```
# TODO: Your code here
def containsA(DNA):

    return ...

# Test your code here.
```

7.3 Write a function called `complement` that returns the complement of the given DNA.

```
# TODO: Your code here
def complement(DNA):
```

```
    return ...

# Test case to check if your code works correctly
print(complement('GATTACA')) # Should be 'CTAATGT'

# Create your own test case.

Ellipsis
```

Exercise 8: Working with Vowels & Consonants []

8.1 Create a list called `vowels` that contains all the vowels from the English alphabet.

```
# TODO: Your code here.
```

8.2 Write a function `printVowels` that iterates through `vowels` and prints each element.

```
# TODO: Your code here.
def printVowels():
    # Note: You can use `vowels` from inside this function.

    return

# Test your code here.
```

8.3 Now, write a function called `isVowel` that takes in a string `letter` and returns `True` if `letter` is a vowel and `False` otherwise. **Do this by looping through vowels.**

```
# TODO: Write your code here.
def isVowel(letter):

    return ...

# Test your code here.
```

8.4 Rewrite a function `isVowel2` that performs the same task as 5.5 but without using a loop.

```
# TODO: Write your code here.
def isVowel2(letter):

    return ...

# Test your code here.
```

8.5 Create a function called `isConsonant`, that returns `True` if the input is not a vowel, and `False` otherwise.

```
# TODO: Write your code here.
def isConsonant(letter):

    return ...

# Test your code here.
```

8.6 Try running the code below. Does it do what you expect?

```
print(isConsonant("a"))
print(isConsonant("b"))
print(isConsonant("1"))
print(isConsonant("."))

Ellipsis
Ellipsis
Ellipsis
Ellipsis
```

8.7 Rewrite the `isConsonant` function so that it also returns `False` when the input character is not part of the alphabet.

Hint: Use string comparators such as `>`, `<`, `>=`, and/or `<=` to test if the character is within the alphabet range.

```
# TODO: Write your code here.
def isConsonant(letter):

    return ...

# Test your code here!
print(isConsonant("a"))
print(isConsonant("b"))
print(isConsonant("1"))
print(isConsonant("."))
```

Exercise 9: List Operations []

9.1 Write a function `inTheList` that takes in a list `lst` and a target element `t` that returns `True` if `t` is in `lst` and `False` otherwise. **Use a for-loop.**

```
# TODO: Write your code here.
def inTheList(lst, t):

    return ...

# Test your code here.
```

9.2 Write a function `overlaps` which takes in two lists, `lst1` and `lst2`, and returns `True` if any element of `lst1` is in `lst2` and `False` otherwise. **You must use `for`-loops.**

```
# TODO: Write your code here.
def overlaps(lst1, lst2):

    return ...

# Test your function here.
```

9.3 Write a function `intersection` which takes in two lists, `lst1` and `lst2`, and returns a list containing the all the elements present in both lists. Hint: use the function from 6.2

```
# TODO: Write your code here.
def intersection(lst1, lst2):

    return ...

# Test your function here.
```

Exercise 10: Palindrome []

10.1 A **palindrome** is a word that is spelt the same forward as it is backwards. For example, 'racecar' backwards is still 'racecar', so it is a palindrome.

For each of the following strings, say whether they are palindromes (in words):

1. "radar"
2. "proper"
3. "aibohphobia"
4. "123123"

10.2 Write a function that takes a string and returns `True` if the second and second-to-last characters of the string are the same and `False` otherwise.

```
# TODO: Write your code here
```

10.3 Write a function that takes a string and returns `True` if the string is a palindrome. Use a `while` loop.

```
# TODO: Write your code here.
def isPalindrome(string):
    ...

    return

# Test your code here.
```

Exercise 11: Function Behavior []

11.1 Look at the function below. Without calling it, can you predict the outcomes of calling `funkyFunc(100)`, `funkyFunc(123456)`, and `funkyFunc(420)`?

```
# Run this cell first.
def funkyFunc(x):
    z = x * 10
    x = x % 10
    z = z + x
    return z

# Check your answers below with code:
...
```

11.2 Can you write a function `funkierFunc` which takes one argument and for which the following equation is true for any $x > 0$:

`funkierFunc( funkyFunc(x) ) == x`

```
# TODO: Your code here.
def funkierFunc(x):
    return ...

# Test your function
if funkierFunc( funkyFunc(123) ) == 123:
    print("CORRECT")
else:
    print("INCORRECT, funkierFunc( funkyFunc(123) ) should be 123")

INCORRECT, funkierFunc( funkyFunc(123) ) should be 123
```

Exercise 12: Prime Number Checking []

We want to define a function that checks if a number is prime.

What is the definition of a prime? In your own words, how would you check if a number is a prime?

In Python, how do you check if a number `n` is divisible by another one (`%` operator)? For example,

- `n = 10` and `a = 5` should give `True`
- `n = 14` and `a = 3` should give `False`
- `n = 1` and `a = 3` should give `False`

```
# TODO: Write an expression here that is True if a divides n and False
otherwise!
n = 10
a = 5
```

Write a function that checks if a number is prime!

Hint: To test if a number `n` is prime, you can check if `n` is divisible by any the number smaller than itself. You use a `while` loop to go through all the numbers smaller than `n`. If you find a number that divides `n` then `n` is not prime.

Hint: Whenever you find a number that divides `n`, `return False`. If you finish the while loop, `return True`.

```
# TODO: Your code here
def is_prime(n):
    pass

assert is_prime(7) == True
assert is_prime(4) == False
assert is_prime(27) == False
assert is_prime(997) == True
assert is_prime(2) == True

print("Test cases passed. ☑")
```

Write a program to print all prime numbers smaller than 100. You should use `is_prime()` defined above.

```
# TODO: Your code here
```

Write a `while` loop that displays the first fifty prime numbers

```
# TODO: Your code here
```

Exercise 13: Digit Ramp []

A digit ramp is a number where the digits increase more often than they decrease when read from left to right. For example, `12343` is a ramp because it has 3 increases (`1->2`, `2->3`, `3->4`) and 1 decrease (`4->3`).

13.1 Implement `ramp`, which takes a non-negative integer `n` and returns `True` if it has more increases than decreases when reading the digits from left to right.

```
# TODO: Write your code here

def ramp(n):
    """Return whether non-negative integer n has more increases than
    decreases"""

    return ...
```

```
# Test your function
print(ramp(12343)) # Should return True
print(ramp(54321)) # Should return False
```

Ellipsis
Ellipsis

Exercise 14: Unique Largest Digit []

14.1 Implement `unique_largest`, a function that takes a positive integer `n` as its input. The function should return `True` if the largest digit in `n` appears exactly once, and `False` otherwise.

Example output:

`unique_largest(132123)` # 3 is the largest digit and appears twice # Output: False

`unique_largest(1321523)` # 5 is the largest digit and appears only once # Output: True

`unique_largest(5)` # 5 is the largest digit and appears only once # Output: True

```
# TODO: Write your code here.
def unique_largest(n):
    """Return whether the largest digit in n appears only once."""
```

```
    return ...
```

```
# Test your code here
```

```
print(unique_largest(132123))
# Output: False
```

```
print(unique_largest(1321523))
# Output: True
```

```
print(unique_largest(5))
# Output: True
```

Ellipsis
Ellipsis
Ellipsis

Exercise 15: Same Digits Comparison []

Implement `same_digits`, a function that takes two positive integers as input. The function should return `True` if both numbers become identical after replacing each sequence of consecutive repeated digits with a single instance of that digit. Otherwise, it should return `False`.

Example:

For the number 12222321, the sequence 2222 would be replaced by a single 2, resulting in 12321.

Restrictions:

In your solution, you may only use combinations of the following elements: a, b, end, 10, %, if, while, and, or, ==, !=, True, False, and return.

Example Outputs:

```
same_digits(2002244, 2202444) # Ignoring repeats, both become 2024
# Output: True

same_digits(21, 12)           # Digits must appear in the same order
# Output: False

same_digits(12, 2212)         # 12 and 212 are not the same
# Output: False

same_digits(2020, 20)         # 2020 and 20 are not the same
# Output: False

# TODO: Write your code here.
def same_digits(a, b):
    """Return whether a and b become the same number after removing
    adjacent repeats."""

    return ...
```

Exercise 16: Writing Functions to Evaluate Lists []

16.1 Write a function `find(lst, x)` that finds the position (index) of `x` in `lst` and prints all elements of the list up to and including `x`. If `x` is not in `lst`, then `find(lst, x)` should return `-1` and print all elements. Use a while loop in your function. For examples:

```
find([1, 3, 4], 3) == 1 # Prints `1` and `3`
find([2, 6, 9, 3], 5) == -1 # Prints all elements

# TODO: Your code here

assert find([1, 3, 4], 3) == 1 # Prints `1` and `3`
assert find([2, 6, 9, 3], 5) == -1 # Prints all elements

print("Test cases passed. ☐")
```

16.2 Write a function `consecutive_sum(lst, x)` that returns `True` if there exists a pair of consecutive elements in `lst` that sum to `x` and `False` otherwise. `consecutive_sum(lst, x)` should also print out the first pair of consecutive elements that sum to `x`. Use a while loop in your function. For example:

```

consecutive_sum([6, 3, 4, 9, 5, 2], 7) == True # Prints `3, 4`
consecutive_sum([2, 8, 9, 3], 23) == False # Prints nothing

# TODO: Your code here

assert consecutive_sum([6, 3, 4, 9, 5, 2], 7) == True # Prints `3, 4`
assert consecutive_sum([2, 8, 9, 3], 23) == False # Prints nothing

print(f"Test cases passed. ☐")

```

Exercise 17: Character Frequency Dictionary ☐

Write a function `letter_freq` that takes a string as input and returns a dictionary. The keys of the dictionary should be the characters in the string, and the values should be the number of times each character appears in the string.

Example:

For the input string "Addiscoder is superfun!", the function should return:

```

{
    "A": 1, "d": 2, "i": 2, "s": 3, "c": 1, "o": 1, "e": 2, "r": 2, " ": 2,
    "u": 2, "p": 1, "f": 1, "n": 1, "!": 1
}

# TODO: Write your code here.
def letter_freq(str1):

    return ...

# Test your code here

```

Congratulations ☐